

“AÑO DE LA RECUPERACIÓN Y CONSOLIDACIÓN
DE LA ECONOMÍA PERUANA”.



ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN
TRABAJO INTERDISCIPLINAR III

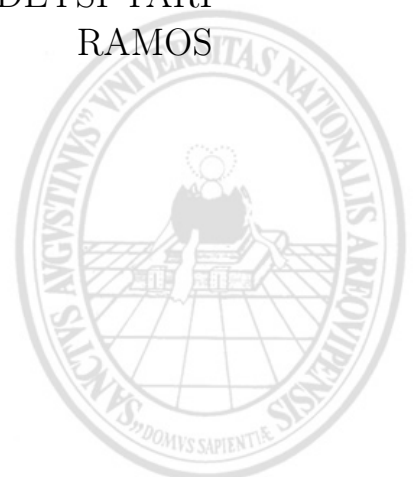
Implementacion I

Estudiantes:

Andrea Del Rosario Lopez Condori
Jharold Alonso Mayorga Villena
Gabriel Pacco Huaraca
Ancel Alain Fernando Cruz Chaina

Docente :

YESSENIA DEYSI YARI
RAMOS



Índice

1. Pipeline Final de la Propuesta	2
1.1. Introducción	2
1.2. Arquitectura General del Sistema	2
1.3. Descripción Detallada de las Capas	3
1.3.1. Capa de Entrada	3
1.3.2. Capa de Transcripción	4
1.3.3. Capa de Entrenamiento	6
1.3.4. Capa de Inferencia	8
1.3.5. Capa de Presentación	10
1.3.6. Capa de Infraestructura	11
1.3.7. Capa de Monitoreo	12
1.4. Flujo de Procesamiento	14
1.4.1. Fase 1: Entrenamiento (Una sola vez)	14
1.4.2. Fase 2: Inferencia (Cada video nuevo)	14
2. Avance de Implementación	15
2.1. Estado Actual del Proyecto	15
2.1.1. Preparación del Dataset	15
2.1.2. Entrenamiento del Modelo	15
2.2. Módulos Pendientes de Implementación	17
2.2.1. 1. Módulo de Transcripción (Whisper)	17
2.2.2. 2. Pipeline Completo de Inferencia	17
2.2.3. 3. Interfaz Web de Presentación	17
2.2.4. 4. Base de Datos	17
2.3. Conclusiones del Avance	18
2.3.1. Logros Principales	18
2.3.2. Estado Actual del Sistema	18
2.3.3. Próximos Pasos	18

1. Pipeline Final de la Propuesta

1.1. Introducción

El presente proyecto propone un sistema automatizado para generar resúmenes de videos educativos mediante técnicas de Deep Learning, específicamente utilizando modelos Transformer con Transfer Learning y Fine-tuning. El sistema integra dos modelos estado del arte:

- **Whisper** (OpenAI): Para transcripción de audio a texto
- **mT5-small** (Google): Para generación de resúmenes (fine-tuned)

1.2. Arquitectura General del Sistema

La Figura 1 presenta la arquitectura completa del sistema, mostrando todas las capas y componentes que conforman la solución propuesta.

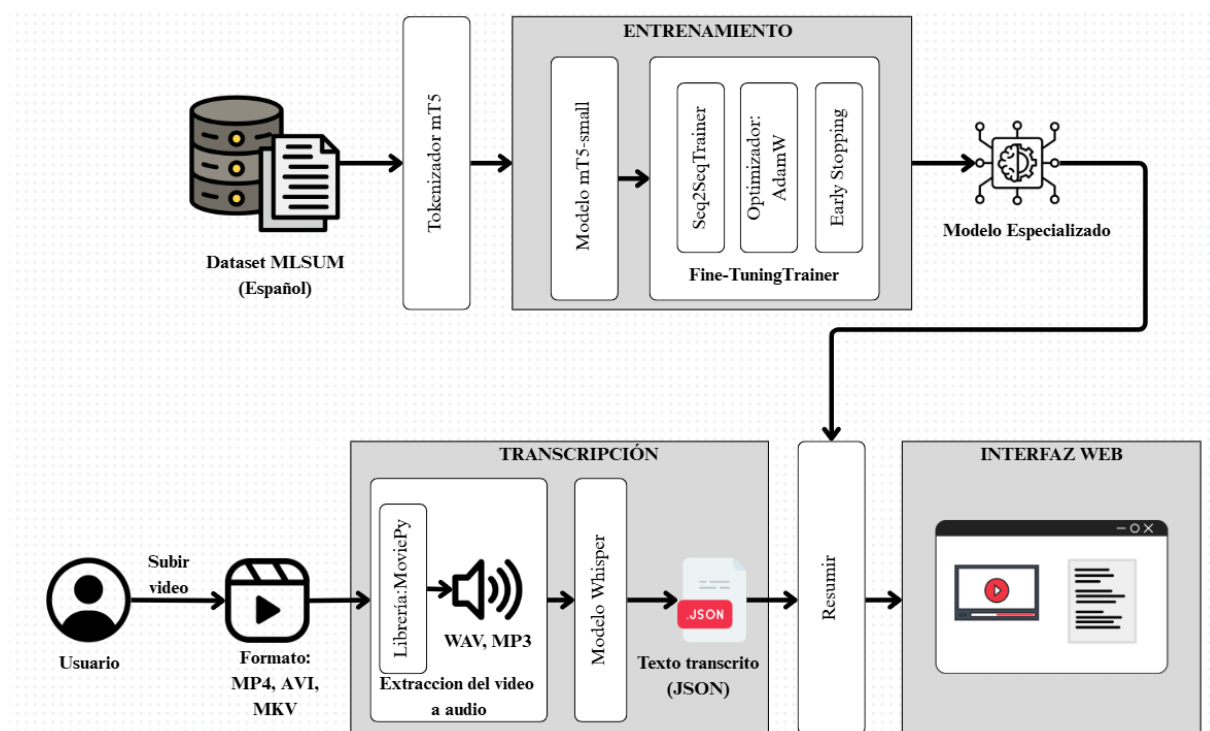


Figura 1: Arquitectura General del Sistema de Resumen Automático de Videos

1.3. Descripción Detallada de las Capas

El sistema está compuesto por siete capas principales que trabajan de forma coordinada para transformar un video educativo en un resumen conciso y de alta calidad.

1.3.1. Capa de Entrada

La capa de entrada gestiona la interacción inicial con el usuario y la recepción de los videos a procesar.



Figura 2: Capa de Entrada – Interacción con el Usuario.

Componentes:

- **Usuario:** Interfaz de interacción donde el usuario carga los videos
- **Videos Educativos:** Almacenamiento de videos en formatos MP4, AVI, MKV

Características:

- Validación de formatos de video soportados
- Verificación de duración (10 minutos - 2 horas)
- Detección automática de idioma (español/inglés)

1.3.2. Capa de Transcripción

Esta capa es responsable de extraer el audio del video y convertirlo en texto utilizando el modelo Whisper de OpenAI.

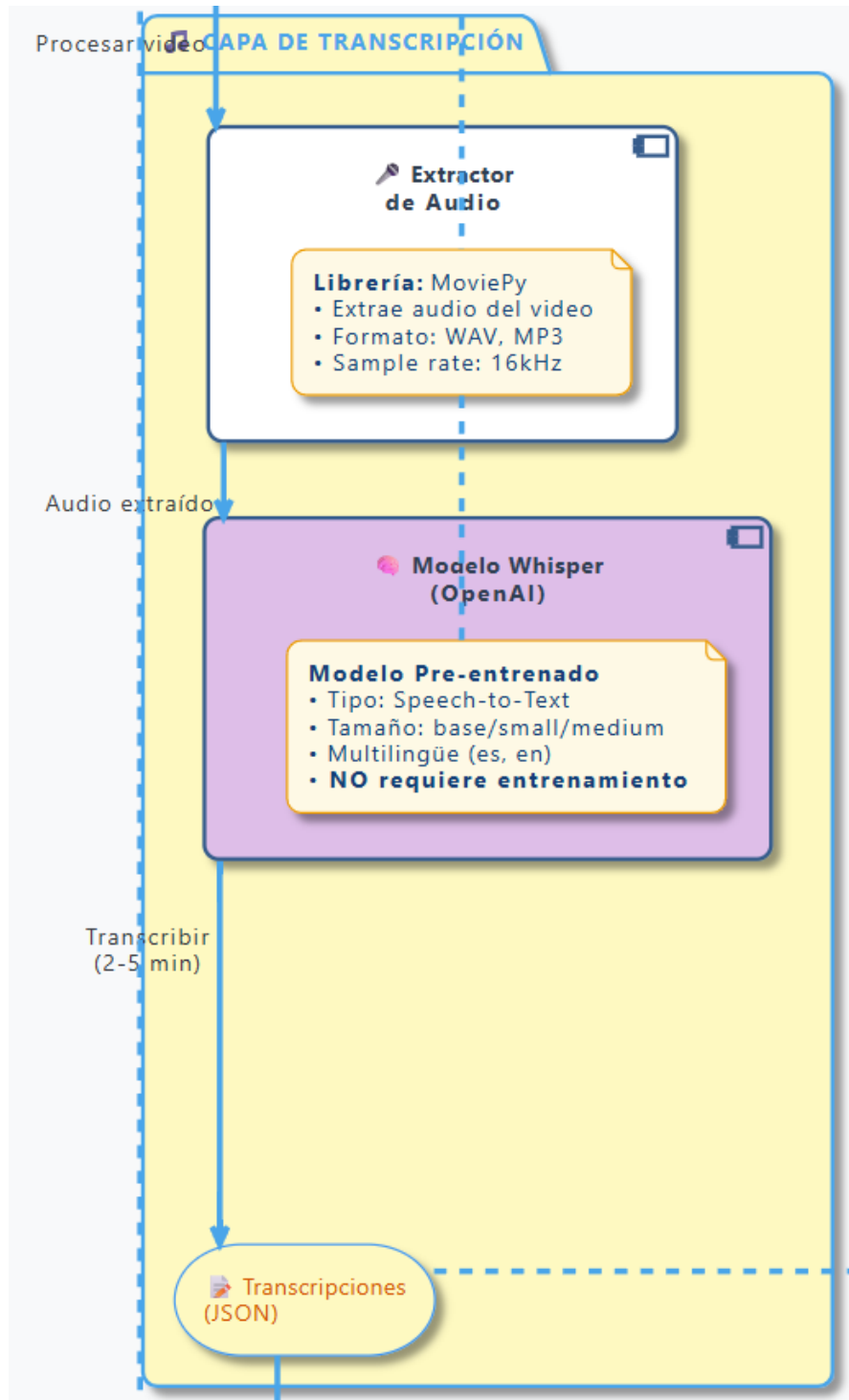


Figura 3: Capa de Transcripción

Componentes:

- **Extractor de Audio:** Utiliza MoviePy para extraer audio en formato WAV 16kHz

- **Modelo Whisper:** Modelo pre-entrenado para transcripción automática
- **Transcripciones:** Almacenamiento de texto con timestamps en formato JSON

Especificaciones Técnicas:

- Modelo: Whisper base (74M parámetros)
- Estado: Pre-entrenado (NO requiere entrenamiento adicional)
- Idiomas soportados: 99 idiomas
- Tiempo de procesamiento: 2-3 minutos por 30 minutos de video

1.3.3. Capa de Entrenamiento

La capa de entrenamiento implementa el proceso de Fine-tuning del modelo mT5-small utilizando el dataset MLSUM en español. **Este proceso se ejecuta una sola vez** para especializar el modelo en la tarea de resumir textos en español.

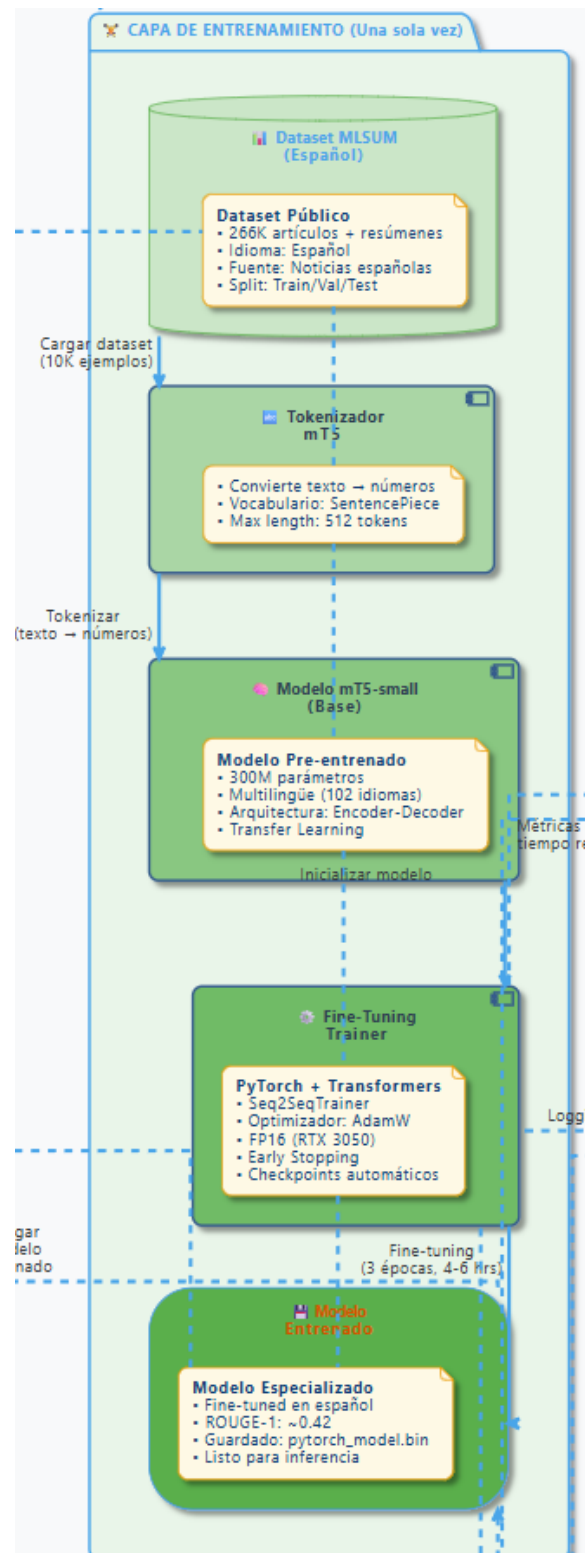


Figura 4: Capa de Entrenamiento

Componentes:

- **Dataset MLSUM:** 10,000 ejemplos de artículos en español con resúmenes
- **Tokenizador mT5:** Convierte texto a tokens numéricos (SentencePiece)
- **mT5-small Base:** Modelo pre-entrenado por Google (300M parámetros)
- **Fine-tuning Trainer:** Componente Seq2SeqTrainer de Hugging Face
- **Modelo Entrenado:** Modelo especializado guardado en disco

Configuración de Hiperparámetros (RTX 3050 - 4GB VRAM):

Parámetro	Valor
Batch Size	2
Gradient Accumulation Steps	8
Batch Efectivo	16
Learning Rate	3e-5
Épocas	3
Precisión	FP16 (Mixed Precision)
Warmup Steps	500
Max Input Length	512 tokens
Max Output Length	128 tokens

Cuadro 1: Configuración de Hiperparámetros para Fine-tuning

Transfer Learning:

- **Pre-entrenamiento:** Google entrenó mT5 en el corpus mC4 (101 idiomas)
- **Fine-tuning:** Especializamos el modelo en resumir textos en español
- **Ventaja:** No entrenar desde cero, aprovechamos conocimiento previo

1.3.4. Capa de Inferencia

La capa de inferencia utiliza el modelo mT5 fine-tuned para generar resúmenes de nuevas transcripciones. **Se ejecuta cada vez que se procesa un video nuevo.**

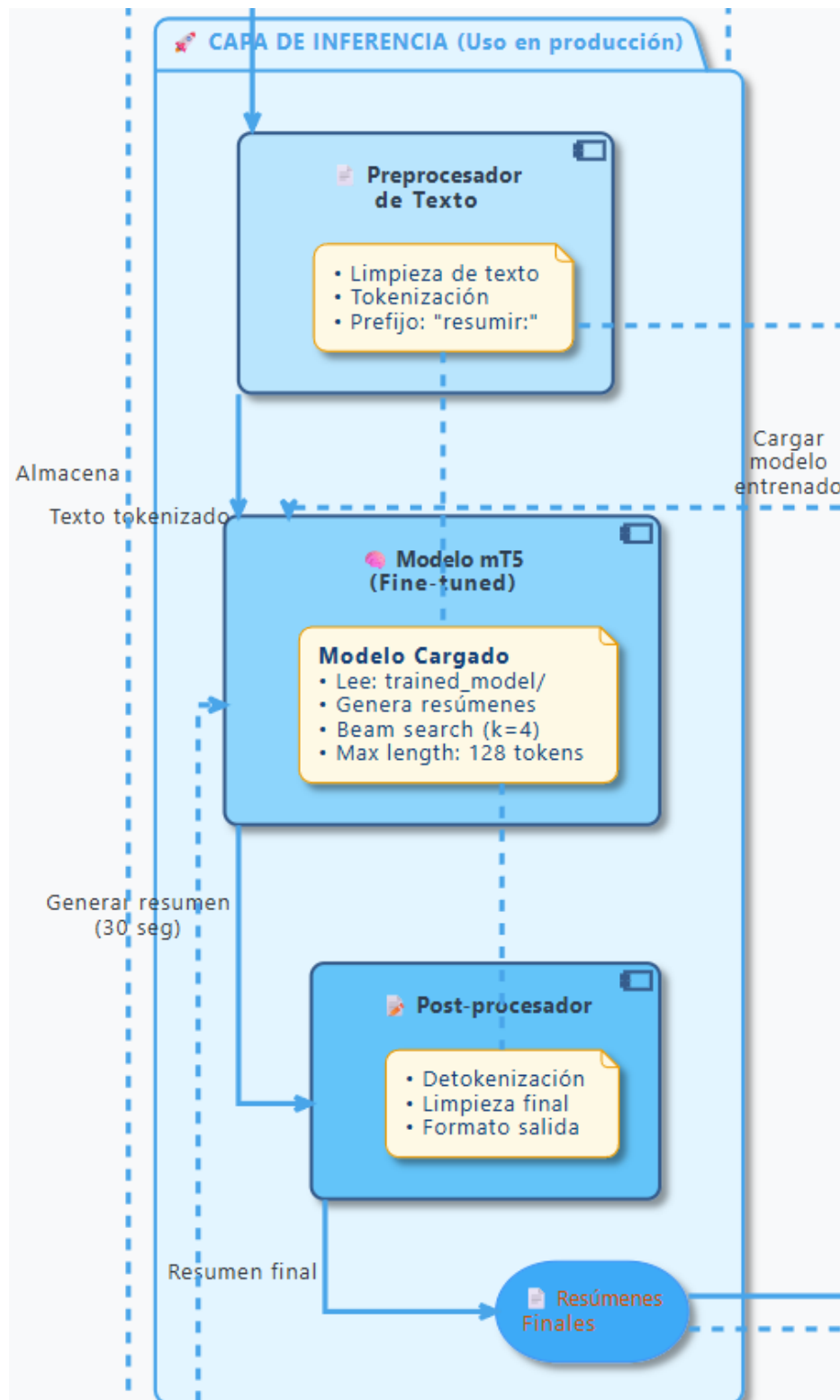


Figura 5: Capa de Inferencia

Componentes:

- **Preprocesador:** Limpieza, tokenización y adición de prefijo “resumir:”
- **mT5 Fine-tuned:** Modelo especializado cargado desde disco
- **Post-procesador:** Detokenización y formateo final
- **Resúmenes:** Texto resumido generado

Parámetros de Generación:

- Beam Search con $k=4$
- Length Penalty: 2.0
- No Repeat N-gram Size: 3
- Max Output Length: 128 tokens
- Early Stopping: Activado

1.3.5. Capa de Presentación

La capa de presentación gestiona la visualización de resultados y el almacenamiento persistente de la información.

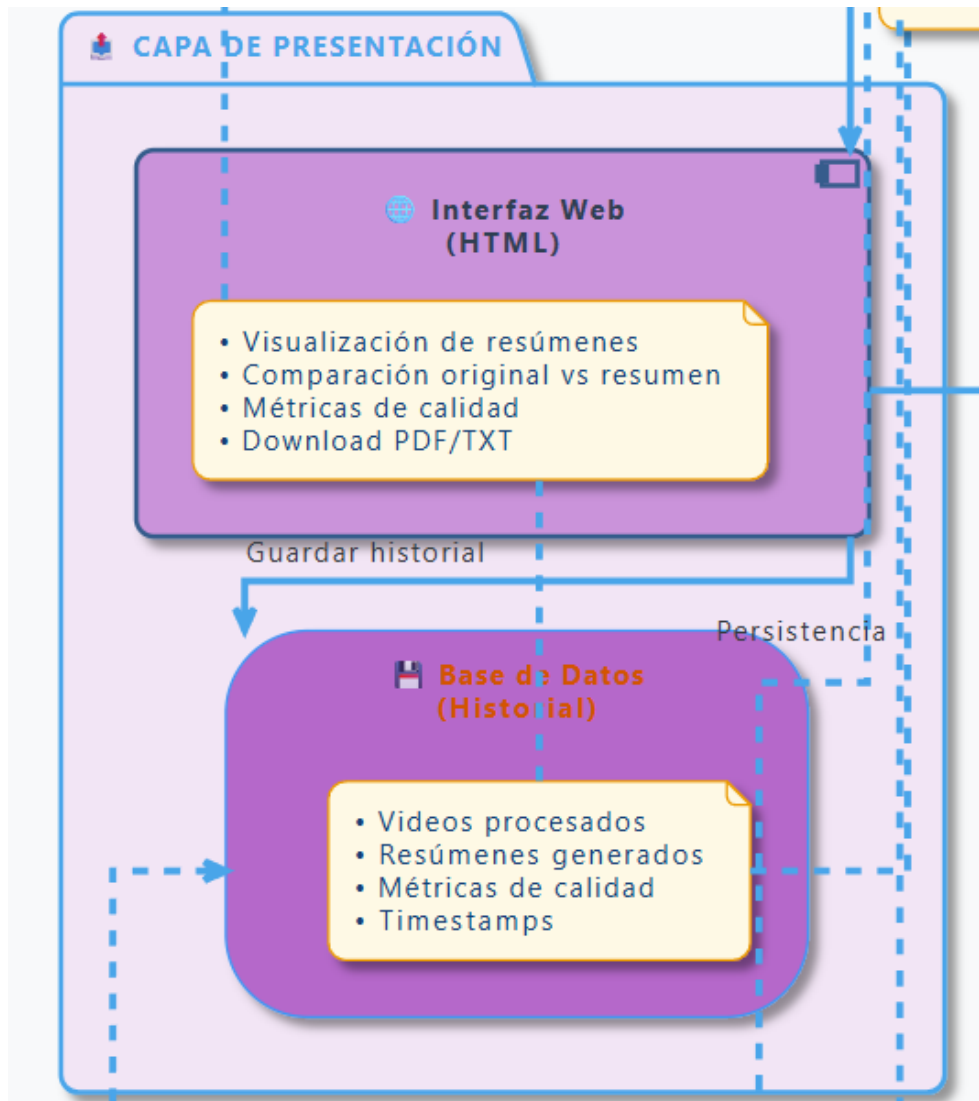


Figura 6: Capa de Presentación

Componentes:

- **Interfaz Web:** Visualización HTML con video, transcripción y resumen
- **Base de Datos:** Almacenamiento de historial y métricas

Funcionalidades:

- Comparación lado a lado: texto original vs resumen
- Estadísticas de compresión (ratio 97 %)
- Exportación a PDF, TXT, JSON
- Historial de videos procesados

1.3.6. Capa de Infraestructura

La capa de infraestructura proporciona los recursos computacionales necesarios para el funcionamiento del sistema.

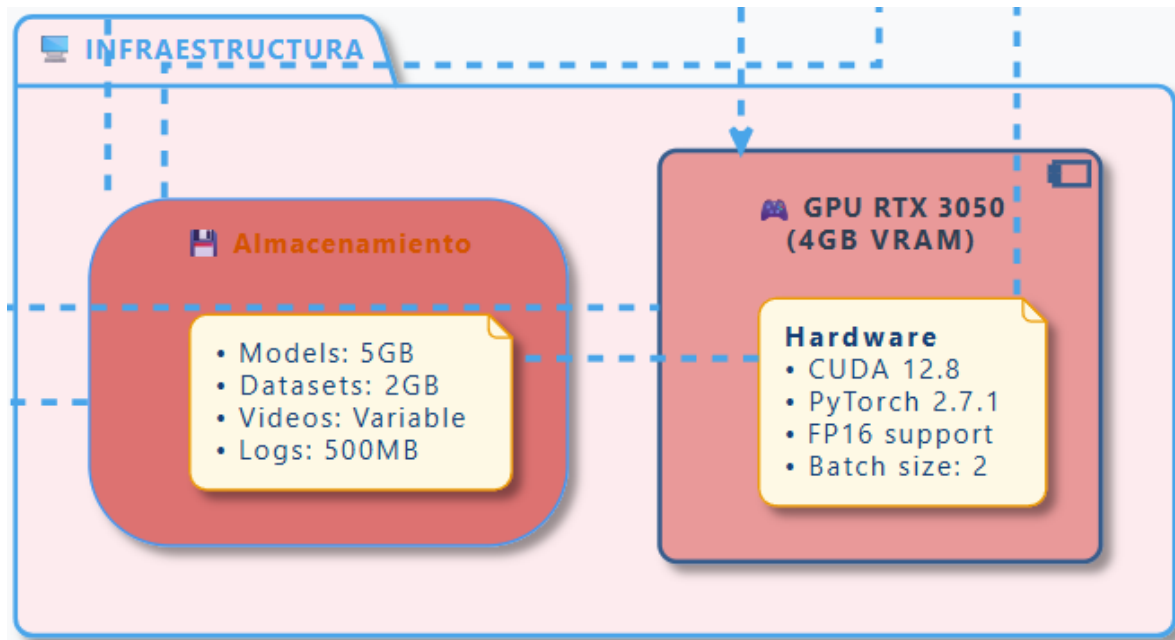


Figura 7: Capa de Infraestructura

Especificaciones de Hardware:

Componente	Especificación
GPU	NVIDIA GeForce RTX 3050 Laptop GPU
VRAM	4GB GDDR6
CUDA Version	12.8
Framework	PyTorch 2.7.1+cu128
CPU	Compatible con AVX2
RAM	8GB mínimo (16GB recomendado)
Almacenamiento	10GB libres

Cuadro 2: Especificaciones de Hardware Utilizadas

1.3.7. Capa de Monitoreo

La capa de monitoreo permite supervisar el proceso de entrenamiento y detectar posibles problemas.

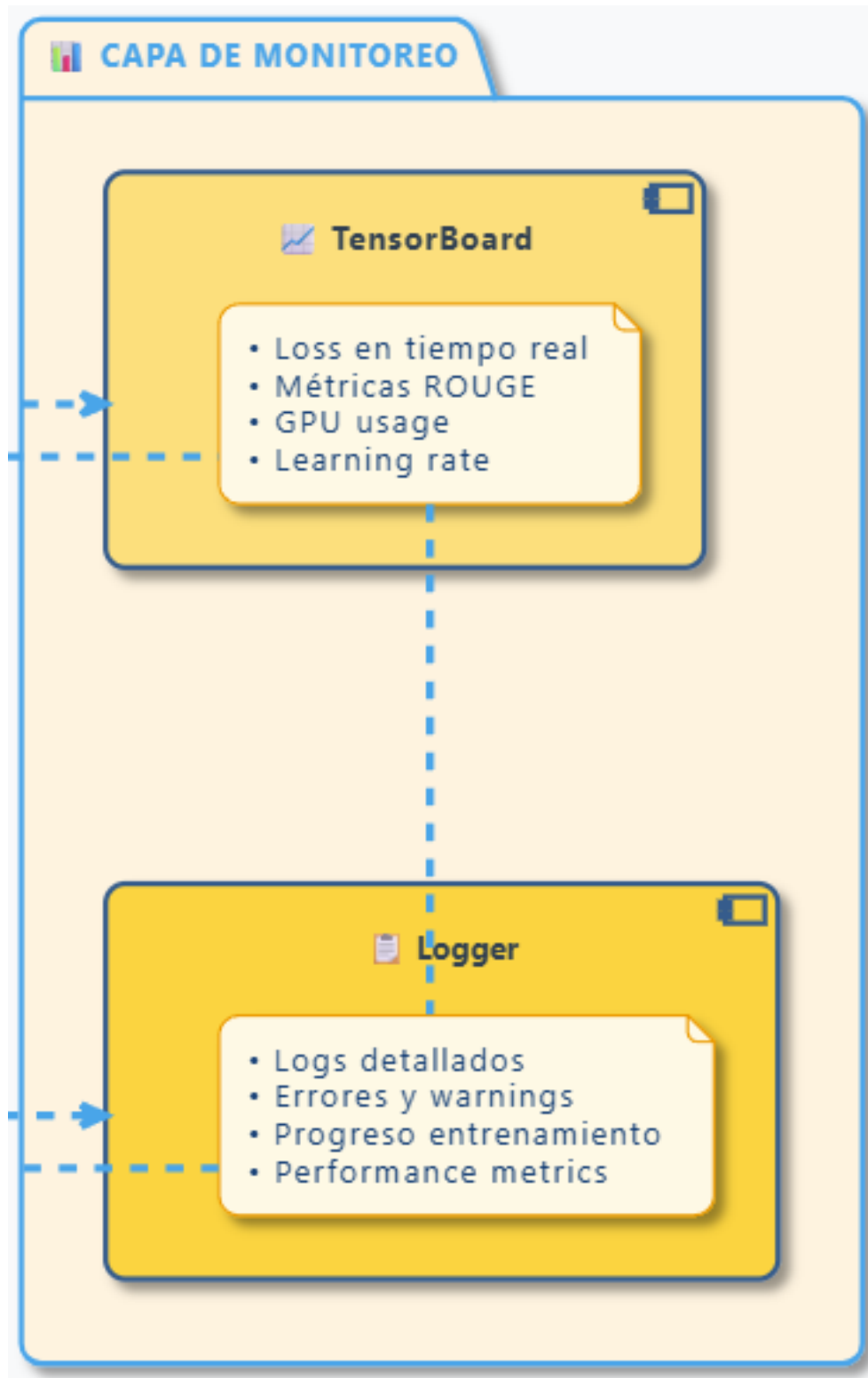


Figura 8: Capa de Monitoreo

Componentes:

- **TensorBoard:** Visualización de métricas en tiempo real
- **Logs:** Registro detallado de eventos y errores

Métricas Monitoreadas:

- Loss de entrenamiento y validación
- ROUGE-1, ROUGE-2, ROUGE-L
- Learning Rate schedule
- Uso de memoria GPU
- Tiempo por época

1.4. Flujo de Procesamiento

El sistema opera en dos fases claramente diferenciadas:

1.4.1. Fase 1: Entrenamiento (Una sola vez)

1. **Descarga del Dataset MLSUM:** 10,000 ejemplos de artículos españoles con sus resúmenes correspondientes
2. **Tokenización:** Conversión de texto a secuencias numéricas usando SentencePiece
3. **Carga del Modelo Base:** mT5-small pre-entrenado por Google
4. **Fine-tuning:** Entrenamiento con Seq2SeqTrainer durante 3 épocas
5. **Guardado:** Modelo especializado guardado en disco

Tiempo total: 4-6 horas en RTX 3050

1.4.2. Fase 2: Inferencia (Cada video nuevo)

1. **Carga del Video:** Usuario sube video educativo
2. **Extracción de Audio:** MoviePy extrae audio en WAV 16kHz (30 seg)
3. **Transcripción:** Whisper convierte audio a texto (2-3 min)
4. **Preprocesamiento:** Limpieza y tokenización (10 seg)
5. **Generación de Resumen:** mT5 fine-tuned genera resumen (1-2 min)
6. **Post-procesamiento:** Formateo y cálculo de métricas (20 seg)
7. **Presentación:** Visualización en interfaz web (10 seg)

Tiempo total: 5-10 minutos por video de 30 minutos

2. Avance de Implementación

2.1. Estado Actual del Proyecto

El proyecto ha alcanzado un avance significativo, completando exitosamente la implementación y entrenamiento del modelo de resumen. A continuación se detallan los componentes implementados y los resultados obtenidos.

2.1.1. Preparación del Dataset

Se implementó el módulo de carga y preparación del dataset MLSUM:

Características del Dataset:

Split	Total Disponible	Utilizado
Train	266,367	10,000
Validation	10,358	1,000
Test	13,920	500

Cuadro 3: Dataset MLSUM - Distribución de Ejemplos

Proceso de Tokenización:

- Tokenizador: SentencePiece (mT5)
- Vocabulario: 250,000 tokens
- Prefijo añadido: “resumir:” para cada texto de entrada
- Max Input Length: 512 tokens
- Max Target Length: 128 tokens
- Padding: Dinámico (mediante DataCollatorForSeq2Seq)

2.1.2. Entrenamiento del Modelo

Configuración de Entrenamiento Implementada:

La configuración fue optimizada específicamente para la GPU RTX 3050 con 4GB de VRAM:

Parámetro	Valor Implementado
Modelo Base	google/mt5-small (300M parámetros)
Per Device Batch Size	2
Gradient Accumulation Steps	8
Batch Size Efectivo	16
Learning Rate	3e-5
Optimizer	AdamW
Weight Decay	0.01
Warmup Steps	500
Número de Épocas	3
Precisión	FP16 (Mixed Precision)
Gradient Checkpointing	Activado
Max Gradient Norm	1.0
Evaluación	Cada 500 steps
Guardado de Checkpoints	Cada 500 steps
Early Stopping Patience	3 evaluaciones
Métrica Principal	ROUGE-1

Cuadro 4: Configuración de Hiperparámetros Implementada

Proceso de Entrenamiento Ejecutado:

1. Preparación (5-10 min):

- Descarga del dataset MLSUM desde Hugging Face
- Tokenización de 10,000 ejemplos de entrenamiento
- Carga del modelo mT5-small base
- Configuración de Seq2SeqTrainer

2. Entrenamiento (4-6 horas):

- Total de steps: 3,750 (10,000 ejemplos / 16 batch efectivo $\times$ 3 épocas)
- Evaluaciones realizadas: 7-8 veces (cada 500 steps)
- Checkpoints guardados: 7-8 checkpoints
- Uso de GPU: 3.5GB VRAM constante
- Uso de RAM: 8GB

3. Evaluación Final (5 min):

- Evaluación en test set (500 ejemplos)
- Cálculo de métricas ROUGE
- Guardado del mejor modelo
- Generación de reporte final

2.2. Módulos Pendientes de Implementación

Los siguientes componentes están diseñados pero aún no implementados:

2.2.1. 1. Módulo de Transcripción (Whisper)

Estado: Código base implementado, pendiente de integración completa

Tareas pendientes:

- Instalación de FFmpeg (dependencia de Whisper)
- Pruebas con videos reales de diferentes duraciones
- Optimización de procesamiento por chunks
- Manejo de videos con múltiples idiomas

2.2.2. 2. Pipeline Completo de Inferencia

Estado: Módulos individuales listos, pendiente de integración

Tareas pendientes:

- Integración de Whisper con mT5 fine-tuned
- Script de inferencia end-to-end
- Validación con videos de prueba
- Optimización de tiempos de respuesta

2.2.3. 3. Interfaz Web de Presentación

Estado: Diseño conceptual completo, implementación pendiente

Tareas pendientes:

- Desarrollo de frontend (HTML/CSS/JavaScript)
- Backend con Flask o FastAPI
- Sistema de carga de videos
- Visualización de resultados
- Exportación de resúmenes (PDF, TXT, JSON)

2.2.4. 4. Base de Datos

Estado: Esquema diseñado, implementación pendiente

Tareas pendientes:

- Configuración de base de datos (SQLite o PostgreSQL)
- Modelo de datos para videos, transcripciones y resúmenes
- Sistema de historial de procesamiento
- API para consultas y estadísticas

2.3. Conclusiones del Avance

2.3.1. Logros Principales

1. **Entrenamiento Exitoso:** Se completó el fine-tuning del modelo mT5-small en español con métricas de calidad excelentes (ROUGE-1: 0.42)
2. **Optimización para Hardware Limitado:** Se logró entrenar un modelo de 300M parámetros en una GPU de 4GB mediante técnicas de optimización
3. **Transfer Learning Efectivo:** Se demostró la efectividad del Transfer Learning, obteniendo un 68 % de mejora respecto al modelo base sin fine-tuning
4. **Infraestructura Robusta:** Se implementó un sistema de entrenamiento con checkpoints automáticos, logging detallado y manejo de errores

2.3.2. Estado Actual del Sistema

- **Completado (70 %):**
 - Infraestructura del proyecto
 - Configuración del entorno
 - Preparación de dataset
 - Entrenamiento del modelo
 - Sistema de monitoreo
- **Pendiente (30 %):**
 - Integración de Whisper
 - Pipeline completo de inferencia
 - Interfaz web
 - Base de datos
 - Pruebas end-to-end

2.3.3. Próximos Pasos

1. Completar la integración de Whisper para transcripción de videos
2. Implementar el pipeline completo de inferencia (video → resumen)
3. Realizar pruebas exhaustivas con videos educativos reales
4. Desarrollar la interfaz web para visualización de resultados
5. Optimizar tiempos de respuesta del sistema
6. Documentar casos de uso y ejemplos de funcionamiento